DPUQuickHMI

Displays a HMI window with contents defined in the QML language.

1. Inputs:

(dynamically defined)	Inputs are dynamically defined using the In parameter list (see below).

2. Static parameters:

MainFile	Main QML file which defines the HMI structure. The file is searched within the project and installation folders. Default: *qml/main.qml*
WindowX, WindowY	Position of the window on screen. Measured in pixels from the top-left edge. Default: 0
WindowW, WindowH	Size of the window, in pixels. Default: 1024 x 768
WindowTopmost	Should the created window always be displayed on top of other windows? Default: false

3. Outputs:

(dynamically defined)	Outputs are dynamically defined using the Out parameter list (see below).

4. Functional principle:

Loads the specified QML file and creates the specified user interface. The window is shown when the simulation is launched; it is hidden when the simulation is stopped.

The user can interact with the user interface using keyboard, mouse and touch in-put. It is possible to transfer data between the HMI and other SILAB DPUs by defin-ing additional inputs and outputs (see below).

5. Defining inputs and outputs:

The In and Out parameter list define which properties in the QML structure are “im-ported from” and “exported to” SILAB. The syntax is as follows:

To import properties from SILAB:

```
In = {
      element1.property1,
```

```
      element2.property2,
      ...
};
```

To export properties to SILAB:

```
Out = {
      element1.property1,
      element2.property2,
      ...
};
```

In both cases, each entry's syntax is *<element id>.<property name>*. That means: the QML **item's id** is followed by a dot, and then the **name** of the **property**.

By default, all these variables are created as floating point numbers. Optionally, string variables can be created, by appending (*string*) after the variable name:

```
In = {
      element1.property1,          # a numeric value
      element2.property2(string)   # a text value
};
```

For example, this can be used to change / retrieve the text of Label and TextInput elements.

Note that signals cannot be exported to SILAB. However, most QML components have an equivalent property that can be exported to SILAB instead. For example, instead of exporting the signal button.clicked, you can export the prop-erty button.pressed. However, you can connect the signals within the QML file and can execute Javascript code when they are emitted.

6. Available QML modules:

Many standard QML modules (which are provided by the Qt framework) are available in SILAB's scope of delivery. Detailed information about them is provided in the following table:

Name and version	Documentation
QtQml 2.2	http://doc.qt.io/qt-5.6/qtqml-qmlmodule.html
QtQuick 2.5	http://doc.qt.io/qt-5.6/qtquick-qmlmodule.html
QtQuick.Controls 1.4	http://doc.qt.io/qt-5.6/qtquick-controls-qmlmodule.html
QtQuick.Dialogs 1.2	http://doc.qt.io/qt-5.6/qtquick-dialogs-qmlmodule.html
QtQuick.Extras 1.4	http://doc.qt.io/qt-5.6/qtquick-extras-qmlmodule.html
QtQuick.Window 2.2	http://doc.qt.io/qt-5.6/qtquick-window-qmlmodule.html
QtGraphicalEffects 1.0	http://doc.qt.io/qt-5.6/qtgraphicaleffects-qmlmodule.html
QtMultimedia 5.6	http://doc.qt.io/qt-5.6/qtmultimedia-qmlmodule.html

They can be used in QML files using the standard import mechanism, for example

```
import QtQuick 2.5
```

Note: Generating or loading resources while the simulation is running can interfer with the execution of the simulation and is therefore not recommended. For the use-case of **exchanging images during the simulation** it is therefore recommended to use multiple instances of the "Image" type. As an alternative the "ImageDisplay" type of SimpleHMI can be used.

7. The QML module SimpleHMI:

In addition to the standard QML modules provided by the Qt framework, SILAB's scope of delivery includes the QML module SimpleHMI, which allows the resource-efficient visualization of typical HMI elements .

An additional benefit is that these QML types are very similar to elements that are available in the DPUMMI. This makes it easy to port existing configurations. The SimpleHMI module can be imported with the following statement:

```
import SimpleHMI 1.0
```

Each provided QML type is described in the following section.

Within the list of its properties, the properties that would usually be imported from SILAB are printed in bold. However, unless noted otherwise, all properties can be imported from SILAB and modified dynamically during the simulation.

8. ImageDisplay:

Corresponds to the ImageDisplay element of DPUMMI.

Displays one image out of a list of images. When entering file names, they are searched starting from the location of the main QML file (the folder containing Main-File), and additionally the directory *SILAB\lib\qml* and the directory

SILAB\lib\graphics\default\mmi.

Properties:

index	Integer that specifies the zero-based index of the currently displayed image within the images list.
images	A list of filenames of images in PNG format. (Other image formats may also be supported by Qt.) Cannot be modified dynamically.

Example:

```
ImageDisplay {
      id: imageDisplay
      images: [   'AS/GearN.png', 'AS/Gear1.png', 'AS/Gear2.png',
                  'AS/Gear3.png', 'AS/Gear4.png', 'AS/Gear5.png' ]
      anchors.left: parent.left
      anchors.leftMargin: 10
      anchors.top: pushButton.bottom
      anchors.topMargin: 10
      width: 150
```

```
        height: 150
        index: 2
}
```

The images shown in the example are loaded from directory *SI-LAB\lib\graphics\default\mmi\AS*.

9. LinearGauge:

Displays a linear gauge instrument or a progress bar. Depending on the current val-ue of the gauge, a part of the *instrumentImage* is displayed on top of the *backgroundImage*.

Properties:

backgroundImage	Filename of the background image
instrumentImage	Filename of the gauge image (or progress bar)
value	Current value to display
valueMin	Minimum displayable value
valueMax	Maximum displayable value
direction	Growth direction of the gauge / progress bar. Use one of the constants Item.Left, Item.Top, Item.Right and Item.Bottom.
xInst, *yInst*	Position of the gauge image within the background image
widthInst, *heightInst*	Size of the gauge image

Example:

The following example implements a fuel level indicator.

```
LinearGauge {
        id: fuel
        backgroundImage: "res/bmwfuel.png"
        instrumentImage: "res/bmwfuellevel.png"
        anchors.left: imageDisplay.right
        anchors.leftMargin: 10
        anchors.top: imageDisplay.top
        anchors.topMargin: 0
        // Size of this element:
        width:255
        height:56
        // Growth direction:
        direction: 5
        // Positioning of the fuel level indicator:
        xInst:31
        yInst:11
        // Valid range of values (liters)
        valueMin: 0
        valueMax: 80
}
```

10. CircularGauge:

Displays a circular gauge instrument. Depending on the current value of the gauge, the needleImage is rotated within the backgroundImage.

Properties:

backgroundImage	Filename of the background image
needleImage	Filename of the needle image
value	Current value to display
xMount, *yMount*	Position of the mounting point of the needle, relative to the top-left corner of the background image
xNeedleMount, *yNeedleMount*	Position of the mounting point of the needle, relative to the top-left corner of the needle image. Note that the needle should point towards the top (12 o'clock position) in its image
angleMin	Minimum angle of the needle (0° = 12 o'clock position)
valueMin	Input value associated with angleMin
angleMax	Maximum angle of the needle (0° = 12 o'clock position)
valueMax	Input value associated with angleMax
angle	Actual angle (not normally necessary to modify)

Example:

The following example implements a speedometer.

```
CircularGauge {
      id: speedo
      backgroundImage: "image://silab/AS/tacho.png"
      needleImage: "image://silab/AS/zeiger.png"
      anchors.left: parent.left
      anchors.leftMargin: 10
      anchors.top: imageDisplay.bottom
      anchors.topMargin: 10
      width:460
      height:360
      // Positioning of the needle
      xMount:226
      yMount:226
      xNeedleMount:9
      yNeedleMount:179
      // Defining the range of values
      valueMin: 0
      valueMax: 200
      angleMin: -126
      angleMax: 126
}
```

11. CircularGaugeExt:

An extended version of the CircularGauge element. Its purpose is to simulate circu-lar gauges in which the lower and upper value range are scaled differently. Most of the properties are the same as in CircularGauge.

Properties:

backgroundImage	Filename of the background image
needleImage	Filename of the needle image
value	Current value to display
xMount, *yMount*	Position of the mounting point of the needle, relative to the top-left corner of the background image
xNeedleMount, *yNeedleMount*	Position of the mounting point of the needle, relative to the top-left corner of the needle image. Note that the needle should point towards the top (12 o'clock position) in its image
angleMin	Minimum angle of the needle (0° = 12 o'clock position)
valueMin	Input value associated with angleMin
angleMed	Middle angle of the needle (0° = 12 o'clock position), the position where its movement speed changes
valueMed	Input value associated with angleMed
angleMax	Maximum angle of the needle (0° = 12 o'clock position)
valueMax	Input value associated with angleMax
angle	Actual angle (not normally necessary to modify)

Example:

The following example implements a speedometer. It shows speeds below 120 km/h precisely, but reduces the precision for higher speeds.

```
CircularGaugeExt {
      id: speedo2
      backgroundImage: "res/opeltacho.png"
      needleImage: "res/opelneedle.png"
      anchors.left: tacho.right
      anchors.leftMargin: 10
      anchors.top: tacho.top
      anchors.topMargin: 0
      width:470
      height:450
      // Positioning of the needle
      xMount:235
      yMount:225
      xNeedleMount:235
      yNeedleMount:225
      // Defining the range of values
      valueMin: 0
      valueMed: 120
      valueMax: 270
```

```
        angleMin: -126
        angleMed: 0
        angleMax: 126
}
```

12. PushButton:

Simulates a push button, which is active while it is pressed down. Two images are used to display the push button in pressed and unpressed state. Additionally, a text can be displayed on the button.

Properties:

text	Text to display on the button
fontSize	Font size of the text
fontFamily	Font family of the text
fontBold	Is the text printed in bold (=1) or normal weight (=0)
fontItalic	Is the text printed in italic (=1) or in oblique (=0)
fontUnderline	Is the text underlined (=1) or not (=0)
textColor	Color of the text
pressedImage	Background image that is displayed while the button is pressed down
unpressedImage	Background image that is displayed when the button is not released
borderWidth	Width of the border area around the text

Outputs:

pressed	Is the button currently pressed down (=1) or not (=0)

Signals:

clicked	The signal is sent when the button is clicked, and can be used within your QML files.

Example:

```
PushButton {
        id: pushButton
        text: qsTr("Hello world!")
        fontSize: 12
        borderWidth: 10
        anchors.left: parent.left
        anchors.leftMargin: 10
        anchors.top: parent.top
        anchors.topMargin: 10
        pressedImage: "res/button-pressed.png"
        unpressedImage: "res/button-unpressed.png"
        //width: 100
}
```

13. ToggleButton:

Simulates a toggle button, which toggles its state whenever it has been pressed and released. Two images are used to display the button in its pressed and its neutral state. By default, the “pressed image” is also used when the button is in the toggled state. However, it is possible to use a third image for this state.

Additionally, a text can be displayed on the button.

Properties:

text	Text to display on the button
fontSize	Font size of the text
fontFamily	Font family of the text
fontBold	Is the text printed in bold (=1) or normal weight (=0)
fontItalic	Is the text printed in italic (=1) or in oblique (=0)
fontUnderline	Is the text underlined (=1) or not (=0)
textColor	Color of the text
pressedImage	Background image that is displayed while the button is pressed down or toggled
unpressedImage	Background image that is displayed when the button is not released
toggledImage	Optional: Background image that is displayed while the button is toggled, but not pressed down)
borderWidth	Width of the border area around the text

Outputs:

toggled	Is the button currently toggled (=1) or not (=0)
pressed	Is the button currently pressed down (=1) or not (=0)

Signals:

clicked	The signal is sent when the button is clicked, and can be used within your QML files.

Example:

```
ToggleButton {
      id: toggleButton
      text: qsTr("Hello world!")
      fontSize: 12
      borderWidth: 10
      anchors.left: pushButton.right
      anchors.leftMargin: 10
      anchors.top: parent.top
      anchors.topMargin: 10
      pressedImage: "res/button-pressed.png"
      unpressedImage: "res/button-unpressed.png"
      toggledImage: "res/button-toggled.png"
      //width: 100
}
```

Example:

The following complex example shows many different QML components and the interaction possibilities using transitions and states.

Use the following DPU script:

```
DPUQuickHMI HMI
{
	Computer =  { LOCALHOST };
	Index = 1;

	WindowX = 0;
	WindowY = 100;
	WindowW = 800;
	WindowH = 800;

	MainFile = "qml/minimal.qml";

	In = {
		progress.value,
		image1.opacity,
		label.text(string)
	};
	Out = {
		slider.value,
		textField.text(string)
	};
};
```

The associated QML file *qml/minimal.qml* is:

```
import QtQuick 2.6
import QtQuick.Window 2.0
import QtQuick.Controls 1.0

Rectangle
{
	id: rectangle

	width: 640
	height: 480

	anchors.fill: parent

	Button {
		id: button
		text: qsTr("Hallo Welt!")
```

```
        anchors.left: parent.left
        anchors.leftMargin: 10
        anchors.top: parent.top
        anchors.topMargin: 10
        opacity: 1.0
    }

    Button {
        id: button1
        x: 247
        y: 311
        text: qsTr("Hallo Mond!")
        visible: false
        anchors.right: parent.right
        anchors.rightMargin: 10
        anchors.bottom: parent.bottom
        anchors.bottomMargin: 10
        opacity: 0.0
    }

    Image {
        id: image
        width: 226
        height: 178
        anchors.left: parent.left
        anchors.leftMargin: 20
        anchors.top: button.bottom
        anchors.topMargin: 20
        fillMode: Image.PreserveAspectFit
        source: "res/biker1.png"
        opacity: 1.0
    }

    Image {
        id: image1
        x: 197
        y: 174
        width: 200
        height: 200
        fillMode: Image.PreserveAspectFit
        anchors.bottom: button1.top
        anchors.bottomMargin: 20
        anchors.right: parent.right
        anchors.rightMargin: 20
```

```
        source: "res/bush1.png"
        opacity: 0.0
    }
    ProgressBar {
        id: progress
        y: 447
        anchors.right: image1.left
        anchors.rightMargin: 10
        anchors.bottom: parent.bottom
        anchors.bottomMargin: 10
        anchors.left: parent.left
        anchors.leftMargin: 10
    }

    Slider {
        id: slider
        y: 412
        anchors.right: image1.left
        anchors.rightMargin: 10
        anchors.bottom: progress.top
        anchors.bottomMargin: 10
        anchors.left: parent.left
        anchors.leftMargin: 10
    }

    Connections {
        target: button
        onClicked: { rectangle.state = "Mond" }
    }

    Connections {
        target: button1
        onClicked: { rectangle.state = "" }
    }
    Connections {
        target: slider
        onValueChanged: progress.value = slider.value
    }

    Label {
        id: label
        width: 156
        height: 13
        text: qsTr("Hallo Welt!")
```

```
        font.pointSize: 18
        font.family: "Verdana"
        anchors.top: image.bottom
        anchors.topMargin: 20
        anchors.left: parent.left
        anchors.leftMargin: 20
}

TextField {
        id: textField
        text: "Test"
        anchors.top: label.bottom
        anchors.topMargin: 20
        anchors.left: parent.left
        anchors.leftMargin: 20
        placeholderText: qsTr("Text Field")
}

transitions: [
        Transition {
                from: "*"
                to: "Mond"

                NumberAnimation {
                        targets: [button, image, button1, image1]
                        properties: "opacity"
                        duration: 1000
                }
        },
        Transition {
                from: "Mond"
                to: ""

                NumberAnimation {
                        targets: [button, image, button1, image1]
                        properties: "opacity"
                        duration: 1000
                }
        }
]
states: [
        State {
                name: "Mond"
```

```
                PropertyChanges {
                      target: button
                      visible: false
                      opacity: 0.0
                }

                PropertyChanges {
                      target: image
                      opacity: 0.0
                }

                PropertyChanges {
                target: button1
                visible: true
                opacity: 1.0
                }

                PropertyChanges {
                      target: image1
                      opacity: 1.0
                }
          }
    ]

}
```